4 IptP serial communications protocol

PREVISTORM® E-Field Sensors support two serial communication protocols: a proprietary plain-text command-response protocol (**IptP**) and a subset of **Modbus RTU** (see section *5*).

The **IptP** protocol is based in the exchange of relatively simple frames that are reasonably easy to read by human readers. It is a text-based protocol where data values are sent in name-value pairs. **IptP** protocol frames can be addressed to "anyone listening" or to a specific node in the bus.

The default serial port configuration in the **IptP** communications protocol is **8-bit**, **No Parity**, **One Stop Bit**, **57600 bps**.

4.1 Framing format

The **IptP** protocol serial communications protocol assumes that multiple devices can coexist in the same serial line. The most common scenario involves only two nodes, the host computer, and the sensor. But installations including the **PREVISTORM® Thunderstorm Warning System** console (MAD) make use of this feature.

IptP protocol frames pack the data to be sent in aliased datasets with a group of name-value pairs per frame. **IptP** frames contain one frame identification header, one data-set name, the list of name-value pairs and, optionally, one 16-bit cyclic redundancy check (CRC) code, see *Table 16*.

	HEADER						CONTENT				CRC			
8	8	8	8	8	8	?	8	?	8	8	8	?	8	8
'#'	*id*	*fi*	'.'	*ti*	*fc*	*alias*	'{'	*params*	'}'	'+'	'('	*c*	')'	'\n'

Greyed fields are OPTIONAL or VARIABLE.

Table 16. Plain-text serial protocol frame fields.

4.1.1 Frame fields explained

'#' Frame start marker.

id Originator local *node id*. Decimal value ranging from 0 to 15 and represented in single-digit hexadecimal format. Default node id uses are:

- Node id 0, ASCII '0', is reserved.
- Node id 1, ASCII '1', is assigned to the Console.
- Node id 2, ASCII '2', is assigned to the Sensor.
- Node id 3, ASCII '3', is assigned to the Host computer.

fi Frame indexing counter. Decimal value ranging from 0 to 15 and represented in single-digit hexadecimal format.

This value is incremented on every frame sent.

'.' ti Target node id [OPTIONAL]. Decimal value ranging from 0 to 15 and represented in single-digit hexadecimal format.

When included in the frame, this field indicates that this frame is destined to one specific node in the bus.

fc Frame class identifier. Identifies the type of frame and can take one of following values:

- ASCII character 'E',
  Identifies a frame that contains asynchronously generated information that represents an event occurring in the system.

- ASCII character 'C',
  Identifies a frame that contains information to be interpreted as a command.

- ASCII character 'R'
  Identifies a frame that contains information to be interpreted as the response to one previous command frame.

alias Data set name. It is used for grouping parameters.

{ Parameters list start delimiter.

params Parameters list. Parameters are sent as <*name*>=<*value*> pairs.

Parameter names have at least one character, starting with one letter and followed by more letters or digits.

Parameters are separated by the character ','.

Supported parameter value types:

- **Integer** (8 to 64-bit).
  Standard ascii representation (base-10).
  The parameter value can be "0" or one decimal digit ranging from '1' to '9' followed by more decimal digits ranging from '0' to '9'.
  Negative numbers start with the character '-'.
  Octal representations are not supported.
  Example: *dummy=-1234567*

- **Unsigned integer** (8 to 64-bit).
  Standard ascii representation (base-10).
  The parameter value can be "0" or one decimal digit ranging from '1' to '9' followed by more decimal digits ranging from '0' to '9'.
  Octal representations are not supported.
  Example: *dummy=U1234567*

- **Hexadecimal** (8 to 64-bit).
  Standard ascii representation (base-16).
  The parameter value starts with letter 'X' followed by one or more hexadecimal digits ranging from '0' to 'F'.
  Example: *dummy=X12345678*

- **Floating point** (32-bit).
  Standard decimal value and engineering floating-point representations.

  Standard decimal floating-point values can be "0" or one decimal digit ranging from '1' to '9' followed by more decimal digits ranging from '0' to '9', followed by character '.' (dot), followed by more decimal digits ranging from '0' to '9'.

  Engineering floating-point values can be followed by the exponent definition: letter 'E' followed by optional sign indicator (character '-') followed by one or more decimal digits ranging from '0' to '9'.
  Negative numbers start with the character '-'.
  Example: *dummy=0.1234*
  Example: *dummy=1.234E-12*

- **String**.
  String values are represented by a string of characters delimited by two quotation mark (") characters denoting the start and end of the string.
  Escape sequences "\r", "\n", "\\", "\"" are interpreted as characters '\r', '\n', '\\' and '\"', respectively.
  Special characters can also be specified using the scape sequence "\u" followed by the character's ascii code in hexadecimal format.
  Example: *dummy="this is a string".*
  Example: *dummy="this string ends here\n".*

- **Switch state**.
  Switch states are represented as one of two possible values: the characters sequence "on", meaning the switch ACTIVE state, and the characters sequence "off", meaning the switch INACTIVE state.
  Example: *dummy=on*.

} Parameters list end delimiter.

'+' Frames chaining. The frames-chaining functionality is used during commands execution for indicating to the receiving node that one response frame is to be followed by at least one more response frame.

'(' CRC value start delimiter. Only present when the optional CRC value is included.

Crc Calculated CRC value represented in hexadecimal format. See section "*Error checking*" below

')' CRC value end delimiter. Only present when the optional CRC value is included.

'\n' Frame end marker.

4.1.2 Error checking

The error checking field is optional. The error checking field markers '(' and ')' are present in the frame only when the CRC value is included.

Error checking utilizes the 16-bit based on standard **ISO3309** Cyclic Redundancy Check (LRC) checksum with initial value 0xFFFF.

The CRC calculation involves all characters from the frame start marker '#' to the frames chaining indication character '+'. The CRC field itself and the frame end marker are not included in the CRC calculation.

4.2 Command/response exchanges

Commands are sent to the **PREVISTORM® E-Field Sensor** encapsulated in frames with the frame class identifier specified as 'C' (command).

Command responses are sent encapsulated in frames with the 'R' (response) frame class identifier.

For commands sent to the sensor the *alias* field is given the meaning of "command name". The reference of valid command names is presented in section *4.4*.

Some commands return more than one group of parameters. Multi-parameter group command responses are formed by chaining two or more response frames. Frame chaining is indicated by including the frame chaining indication flag ('+' symbol) after the parameters list end delimiter ('}').

All command responses include at least two parameters, named "CFID" and "CEDC". One serves for identifying the command to which the response is associated, and the other contains the resulting code for the operation performed by the command.

The first frame in the response frames chain contains the integer parameter named "CFID". This parameter indicates the value of the frame indexing counter (*fi* field) received in the command frame.

The last frame in the response frames chain contains the integer parameter named "CEDC". This parameter contains the command execution result description code. Valid execution result codes are listed in *Table 17*, below. The last frame always contains the same data-set alias value as the command frame. Note that this behaviour can force the generation of one additional frame, which will be the last frame, for some commands that generate response frames with different data-set names.

Code	Value	Description
SUCCESS	0	Command execution completed successfully.
UNRECOGNIZED_COMMAND	1	Unrecognized command alias.
FRAME_TOO_LONG	2	Command frame is too long, command was discarded.
OUT_OF_MEMORY	4	Not enough memory to complete the requested operation.
ARGUMENT_OUT_OF_RANGE	5	At least one command argument, frame parameter, has an out-of-range value.
INVALID_OPERATION	6	The requested operation is not allowed.
ARGUMENT_FORMAT_ERROR	7	At least one command argument, frame parameter, has an invalid format.
ARGUMENTS_ERROR	8	Other unspecified argument validation error detected or at least one mandatory argument is missing.

Table 17. Command execution result description codes.

Response type frames are always addressed directly to the node that issued the command. This is done by including the field '.' *ti* (target node identifier) described earlier in section *4.1.1*.

4.3 Asynchronous events

The **PREVISTORM® E-Field Sensor** sends asynchronous events encapsulated in two possible frame formats: using grouped name-value pairs, like in the command-response framing format, see section *4.1*, or within a fixed-length, plain-text frame format, see section *4.3.2*.

The format used for the asynchronous events is configured with the command "`comm`", see section *4.4.3*.

4.3.1 Name-value pairs event frame format

When sent as name-value pairs, event frames have the frame class identifier specified as '**E**' (event). The alias frame field identifies the module to which the event is relevant.

Section *4.5* contains the reference of asynchronous events sent by the sensor when grouped using name-value pairs.

4.3.2 Fixed-length plain-text event frame format

When using the fixed-length, plain-text frame format, no **IptP** frames are sent. All fields in this asynchronous events frame format have fixed positions and lengths.

The following table summarises the frame fields format:

Offset	Length	Format	Description
0	1	Fixed value: '$'	Frame start marker
1	1	Hex digit	*Sender's Node Id*.
2	1	Hex digit	*Frame Index*.
3	1	Fixed value: '1'	Frame format Id.
4	1	Fixed value: ','	Separator.
5	8	Hex unsigned long	*Local timestamp*.
13	1	Fixed value: ','	Separator.
14	2	Right aligned decimal	*Application status*.
16	1	Fixed value: ','	Separator.
17	1	Decimal digit	*EFValid flag*.
18	1	Fixed value: ','	Separator.
19	2	Right aligned decimal	*Current alarm level*.
21	1	Fixed value: ','	Separator.
22	6	Right aligned decimal	*Measured electric field strength*.
28	1	Fixed value: ','	Separator.
29	4	Hex digits	*Output relays status* flags
33	1	Fixed value: ','	Separator.
34	5	Right aligned floating point. One digit after decimal point	*Main board's temperature* in ºC.
39	1	Fixed value: ','	Separator.
40	5	Right aligned floating point. One digit after decimal point	*Sensor's base temperature* in ºC.
45	1	Fixed value: ','	Separator.
46	5	Right aligned floating point. One digit after decimal point	*Sensor's head temperature* in ºC.
51	1	Fixed value: '('	CRC16 start marker.
52	4	Hex unsigned short	*CRC16*.
56	1	Fixed value: ')'	CRC16 end marker.
57	1	Fixed value: '\n'	Frame end marker.

Sender's Node Id:

Decimal value ranging from 0 to 15 and represented in single-digit hexadecimal format. Same as the *Originator local node id*.

Frame Index:

Decimal value ranging from 0 to 15 and represented in single-digit hexadecimal format. Follows the same sequence of the *Frame indexing counter*.

Local timestamp:

Internally generated 32-bit timestamp with millisecond resolution.

Application status:

Decimal value ranging from 0 to 8. Represents the current application status.

Possible values are listed on page *86*.

EFValid flag:

Decimal value ranging from 0 to 1. Represents the electric field measurements status.

When the fields *Measured electric field strength* and *Current alarm level* have valid values, this field has value 1, 0 otherwise.

Current alarm level:

Decimal value ranging from -1 to 3. Represents the current alarm level reported by the sensor.

This field is right-aligned, left-padded with space characters (' ').

When the field *EFValid flag* has value 0, this field must be ignored and have value -1.

When the field *EFValid flag* has value 1, this field contains the current alarm level reported by the sensor. Value 0 represents the **NO_ALARM** alarm level and values 1 to 3 represent the alarm levels **LEVEL_1** to **LEVEL_3**, respectively.

Measured electric field strength:

Decimal value ranging from -99999 to 99999. Represents the value of the measured electric field intensity in **V/m**.

This field is right-aligned, left-padded with space characters (' ').

When the field *EFValid flag* has value 0, this field must be ignored and is empty.

When the field *EFValid flag* has value 1, this field contains the value of the measured electric field intensity in **V/m**.

Output relays status:

4-digits hexadecimal value where each digit can have values 0 or 1. Each digit represents a relay status flag.

When a status flag has value 0, the corresponding relay is in deenergised, **OFF**, state.

When a status flag has value 1, the corresponding relay is in energised, **ON**, state.

The digits, listed from LSB to MSB, represent the state of the output relays **SIGNAL-1**, **SIGNAL-2**, **Base-Heater** and **Sensor Heater**, respectively.

Main board's temperature:

Floating point value ranging from -99.9 to 99.9. Represents the main board's temperature in ºC.

This field is right-aligned, left-padded with space characters (' ').

Temperature measuring and monitoring is only available on sensors model **700024**.

This field is empty when the temperature measuring module is not installed in the sensor, or when it is failing.

Sensor's base temperature:

Floating point value ranging from -99.9 to 99.9. Represents the temperature at the sensor's base in ºC.

This field is right-aligned, left-padded with space characters (' ').

Temperature measuring and monitoring is only available on sensors model **700024**.

This field is empty when the temperature measuring module is not installed in the sensor, or when it is failing.

Sensor's head temperature:

Floating point value ranging from -99.9 to 99.9. Represents the temperature at the sensor's head in ºC.

This field is right-aligned, left-padded with space characters (' ').

Temperature measuring and monitoring is only available on sensors model **700024**.

This field is empty when the temperature measuring module is not installed in the sensor, or when it is failing.

CRC16:

4-digit hexadecimal value ranging from 0000h to FFFFh. Represents the Cyclic Redundancy Check (LRC) checksum calculated on the frame.

The CRC calculation is made over all characters ranging from the frame start marker '$' to the last character before the CRC16 field start marker '('. The error checking field markers '(' and ')' and the CRC field are not included in the CRC calculations.

The CRC16 calculation is done based on the standard **ISO3309** Cyclic Redundancy Check (LRC) checksum with initial value FFFFh.

4.4 Sensor commands reference

4.4.1 "adj" - Analog front-end access command

The "**adj**" command gives access to the analog front-end configuration for reading and modifying the *Form_Factor* and other gain-related parameters. Changes made with this command are temporary and, if required, can be made permanent by invoking the command "**scfg**", see section *4.4.13*.

The response dataset named "**adj**" contains the sensor calibration values. The parameters encapsulated in this dataset are listed in the following table.

Dataset	Parameter	Type	Access	Description
adj	**cf**	**float**	R/W EXPERT	*Conversion Factor*. Global digital gain. This parameter is adjusted during calibration. Its modification by the user will invalidate the sensor's calibration.
	ff	**float**	R/W BASIC	*Form Factor*. Site calibration form factor, see sections *2.5* and *2.6*.
	sofr	**float**	R/W EXPERT	*Sensor Offset (raw)*. Offset correction parameter. This parameter is adjusted during calibration. Its modification by the user will invalidate the sensor's calibration.
	std	**long**	R/W EXPERT	*MEC-1*. Mechanical calibration 1. This parameter is adjusted during calibration. Its modification by the user will invalidate the sensor's calibration.
	stp	**long**	R/W EXPERT	*MEC-2*. Mechanical calibration 2. This parameter is adjusted during calibration. Its modification by the user will invalidate the sensor's calibration.

Usage example (pseudo-code, CRC and target fields not included):

Set the site calibration form factor parameter to 0.15.

```
#31Cadj{ff=0.15}\n
```

From the host (**3**), frame index (**1**), Command (**C**) Adjust (**adj**), set the *Form Factor* (**ff**) parameter to **0.15**.

The response dataset named "**gainset**" contains information about the analog front-end gains supported by the signal chain. The parameters encapsulated in this dataset are listed in the following table.

Dataset	Parameter	Type	Access	Description
gainset	**idx**	**long**	R	*Gain Index*. Logical index of this gain.
	ofs	**float**	R	*Current Offset Correction*. Offset correction factor for the auto-gain/auto-offset correction protocol.
	gain	**float**	R/W EXPERT	*Gain Value*. Calibrated gain value of this analog gain step. This parameter is adjusted during calibration. Its modification by the user will invalidate the sensor's calibration.

The response dataset named "**filter**" contains information about the digital filter which is applied for reducing noise at the digital output. The parameters encapsulated in this dataset are listed in the following table.

Dataset	Parameter	Type	Access	Description
filter	**typ**	**string**	R	*Filter Type*. Type of the filter. Supported values are: "**mvavg**" - Moving average filter.
	mxtaps	**long**	R	*Maximum Filter Taps*. Maximum number of supported filter taps for the moving average filter.
	taps	**long**	R/W EXPERT	*Moving Average Filter Taps*. Number of filter taps for the Moving average filter. After modifying this parameter, the sensor will stop producing output values until all the filter taps are filled with valid data.

Usage example (pseudo-code, CRC and target fields not included):

Set the number of active taps for the output moving average filter.

```
#31Cfilter{taps=2}\n
```

From the host (**3**), frame index (**1**), Command (**C**) Filter (**filter**), set the *Moving Average Filter Taps* count (**taps**) parameter to **2**.

4.4.2 "alarm" - Alarms generation engine access command

The "**alarm**" command gives access to the alarm generation engine configuration. Changes made with this command are temporary and, if required, can be made permanent by invoking the command "**scfg**", see section *4.4.13*. This command supports resetting current alarm level to 0.

When this command is invoked without parameters the command response from the sensor will contain one dataset named "**alarm**" for each alarm level.

For requesting the parameters of one specific alarm level the command should be invoked specifying the alarm level in the parameter "a".

Dataset	Parameter	Type	Access	Description
alarm	**a**	**long**	R	*Alarm Index*. Logical alarm index, starting in Alarm Level "1".
	t	**float**	R/W BASIC	*Threshold*. Alarm threshold value.
	d	**ulong**	R/W BASIC	*Deactivation delay*. Alarm level dwell time (DT), see section *2.2.2*.

For resetting current alarm level to 0 the command should be invoked specifying one single integer parameter named "oper" with value "**1**".

Dataset	Parameter	Type	Access	Description
alarm	**oper**	**int**	R	*Operation code*. Should be "0" for resetting current alarm level.

Usage example (pseudo-code, CRC and target fields not included):

Set the alarm LEVEL_1 threshold to 2.5kV.

```
#31Calarm{a=1,t=2.5E3}\n
```

From the host (**3**), frame index (**1**), Command (**C**) Alarm (**alarm**), set alarm level *Threshold* (**t**) parameter to **2.5kV**.

4.4.3 "comm" – Communications protocol configuration command

This command is supported in firmware versions **1.030** or newer.

The "**comm**" command gives access to the communications protocol configuration. Changes made with this command are temporary and, if required, can be made permanent by invoking the command "**scfg**", see section *4.4.13*.

The changes made to the communications protocol configuration will take effect on next sensor boot. See command "reset" in section *4.4.12*.

When this command is invoked without parameters, the command response from the sensor will contain one dataset named "**comm**" with the current parameter values for controlling the communications protocol.

Dataset	Parameter	Type	Access	Description
comm	**prot**	**ulong**	R/W **BASIC**	*Communications protocol*. Current communications protocol.
	mba	**ulong**	R/W BASIC	*Modbus: Address*. Node address used when operating in **Modbus RTU** protocol mode. Valid range: **1-247**. Default: **11**.
	mbb	**ulong**	R/W BASIC	*Modbus*: *Bitrate*. Bitrate, in bps, used when operating in **Modbus RTU** protocol mode.
	pto	**ulong**	R/W BASIC	*Plain-text Protocol Options* Protocol control options, used when operating in plain-text protocol mode (**IptP**).

The following table summarises the supported communications protocols:

prot	Communications mode
0	Plain-text serial (default).
1	Modbus RTU.

The following table summarises the supported bit rates for the **Modbus RTU** mode:

Mbb Bitrate
9600
14400
19200
38400
57600

The registers memory map and protocol support for operation in **Modbus RTU** mode are covered in section *5*, *Modbus RTU serial communications protocol*.

The following table summarises the supported plain text protocol options:

Bit	Option
0	Event frames format. 0: Use IptP frames format, see section *4.3.1*. 1: Use fixed length frames format, see section *4.3.2*.
All other bits are reserved and should be specified with value '0'.	

4.4.4 "app" - Application status access command

The "**app**" command requests the current application state to the sensor. This command expects no input parameter.

The response dataset named "**app**" contains information about current application state. The parameters encapsulated in this dataset are listed in the following table.

Dataset	Parameter	Type	Access	Description
app	state	string	R	*Application State (text)*. Text representation of the current application state. See table below.
	s	long	R	*Application State (integer)*. Binary representation of the current application state. See table below.
	omr	bool	R	*Operating Mode Requested*. Flag indicating that the sensor received the request to enter operational mode.

The following table lists the text and binary values representing current application states:

Binary	Text	Description
0	UNINITIALIZED	Initialization in progress.
1	IDLE	System idling.
2	CALIBRATION_WAITING_MOTORSTOP	Running internal calibration.
3	CALIBRATION_WAITING_BEFORE_START	
4	CALIBRATING	
5	WAITING_BEFORE_AUTOSTART	Pausing before starting the sensor.
6	PRE-OPERATIONAL	Starting the sensor.
7	OPERATIONAL	Sensor running.
8	HWERROR	Hardware error condition.

4.4.5 "heater" - Heaters control engine access command

The "**heater**" command gives access to the heaters control engine configuration. This command only applies to sensor model **PREVISTORM® E-Field Sensor 700024**. Changes made with this command are temporary and, if required, can be made permanent by invoking the command "**scfg**", see section *4.4.13*.

When this command is invoked without parameters the command response from the sensor will contain one dataset named "**baseheater**" with the current parameter values for the reference plate heater controller. The dataset named "**sensorheater**" will contain the current parameter values for the sensing plate heater controller.

Dataset	Parameter	Type	Access	Description
heater	m	long	R/W EXPERT	*Controller Operating Mode*. Operating mode for the controller. Supported values are: "0": None (No heaters). "1": Thermostat mode.
	e	bool	R/W ADVANCED	*Controller Enabled*. Flag indicating that the heater controller is enabled.
	mno	float	R/W ADVANCED	*Minimum Operational Temperature*. For temperatures below the minimum operational temperature the heater controller will be automatically disabled.
	mxo	float	R/W ADVANCED	*Maximum Operational Temperature*. For temperatures above the maximum operational temperature the heater controller will be automatically disabled.
	up	long	R/W ADVANCED	*Update period*. Controller update period, in milliseconds.
	on	float	R/W ADVANCED	*Thermostat Activation Temperature*. For temperatures below the activation temperature the heater controller will activate the heater.
	off	float	R/W ADVANCED	*Thermostat Deactivation Temperature*. For temperatures above the activation temperature the heater controller will deactivate the heater.

Usage example (pseudo-code, CRC and target fields not included):

Set the heater controller Activation and Deactivation temperatures for the thermostat mode to 1ºC and 5ºC, respectively.

```
#31Cheater{on=1.0,off=5.0}\n
```

From the host (`3`), frame index (`1`), Command (`C`) Alarm (`heater`), set *Activation Temperature* (`on`) parameter to 1.0 and *Deactivation Temperature* (`off`) parameter to 5.0.

4.4.6 "motor" - Motor control engine access command

The "`motor`" command gives access to the motor control engine configuration. Changes made with this command are temporary and, if required, can be made permanent by invoking the command "`scfg`", see section *4.4.13*.

When this command is invoked without parameters the command response from the sensor will contain one dataset named "`motor`" with the current state values of the motor controller.

Dataset	Parameter	Type	Access	Description
`motor`	`gpio`	`switch`	`R`	*Motor ON state*. Current state of the hardware motor enable control.
	`ctrl`	`string`	`R`	*Current Controller State (text)*. Text representation of the current motor controller state. See table below.
	`rpm`	`long`	`R`	*Current Motor Speed*. Current motor turning speed in revolutions per minute.
	`i`	`float`	`R`	*Motor Current Consumption*. Current motor current consumption measured by the motor controller.

The following table lists the text and binary values representing current motor controller states:

Text	Description
IDLE	Controller is idling.
ACCELERATING1 **ACCELERATING2**	The motor is accelerating (motor start requested).
RUNNING	Motor running and ready.
DECELERATING	The motor is decelerating (motor stop requested).
OVERCURRENT	Over current condition detected.
POWERERROR	Motor power source related error condition, typically related with cable errors.
MOTORERROR	Generic motor error condition.

4.4.7 "start" - Measurements engine control access command

The "**start**" command request the system to start operation.

This command is invoked without parameters.

4.4.8 "stop" - Measurements engine control access command

The "**stop**" command request the system to stop operation.

This command is invoked without parameters.

4.4.9 "who" - Generic system identification request command

The "**who**" command request information about the system.

When this command is invoked without parameters the command response from the sensor will contain one dataset named "**who**" with parameters describing the current application running in the sensor.

Dataset	Parameter	Type	Access	Description
who	**name**	**string**	**R**	*Running Application Name*. Descriptive name of the currently running application.
	rev	**string**	**R**	*Running Application Revision*. Revision code of the currently running application.

4.4.10 "notif" - Messages generation engine control access command

The "**notif**" command gives access to configuration parameters for the application module that generates the events and notifications sent by the sensor.

Changes made with this command are temporary and, if required, can be made permanent by invoking the command "**scfg**", see section *4.4.13*.

When this command is invoked without parameters the command response from the sensor will contain one dataset named "**notif**" containing the enabled notifications mask and the currently configured notifications generation period.

Firmware versions: up to **1.025**:

Dataset	Parameter	Type	Access	Description
notif	**msk**	**long**	R/W ADVANCED	*Enabled Notifications Mask*. Binary mask containing one bit for each enabled notification. See table below.
	int	**long**	R/W EXPERT	*Notifications interval*. Time interval between two consecutive notifications.

The following table lists the notifications bit mask:

Bit	Notification
0	Motor current consumption and speed.
1	Application state changes.
2	Motor controller state changes.
3	Temperatures.
4	RESERVED, should always be specified as 0.
5	Heater controllers state changes.
6	Timestamp all notifications with one internally generated milliseconds resolution timestamp.
7	RESERVED, should always be specified as 0.

Firmware versions: from **1.026**:

Dataset	Parameter	Type	Access	Description
notif	**msk**	**long**	R/W ADVANCED	*Enabled Notifications Mask*. Binary mask containing one bit for each enabled notification. See table below.
	pdp	**long**	R/W EXPERT	*Timing granularity*. Time base, in **ms**, used for handling the notifications. 50ms ≤ **pdp** ≤ 1000ms.
	etp	**long**	R/W EXPERT	*Electric field strength notifications periodicity.* Period used for sending values of electric field strength, specified in multiples of parameter **pdp**. 100ms ≤ **etp** * **pdp** ≤ 2000ms.
	ttp	**long**	R/W EXPERT	*Temperature notifications periodicity.* Period used for sending values of temperature, specified in multiples of parameter **pdp**. 250ms ≤ **ttp** * **pdp** ≤ 5000ms.
	mtp	**long**	R/W EXPERT	*Motor consumption and speed notifications periodicity.* Period used for sending values of motor consumption and speed data, specified in multiples of parameter **pdp**. 100ms ≤ **mtp** * **pdp** ≤ 5000ms.

4.4.11 "sgnlng" - Signalling outputs control engine access command

The "**sgnlng**" command gives access to the signalling engine configuration. Changes made with this command are temporary and, if required, can be made permanent by invoking the command "**scfg**", see section *4.4.13*.

When this command is invoked without parameters the command response from the sensor will contain one dataset named "**sgnlng**" with information about the signalling engine implementation, one dataset named "**insts**" containing the binary code for the signalling instructions and one dataset named "**seq**" for each configured signalling sequence.

Dataset	Parameter	Type	Access	Description
sgnlng	**mi**	**long**	R	*Maximum Instructions Count*. Maximum number of instructions supported.
	ms	**long**	R	*Maximum Sequences Count*. Maximum number of sequences supported.
insts	**lst**	**blob**(uint32)	R	*Binary Coded Instructions*. Signalling engine control instructions in binary format. Each 32-bit unsigned integer represents one signalling control instruction.
seq	**idx**	**ulong**	R	*Signalling Pattern Index*. Index of the signalling pattern. This index is used also by the signalling control instructions.
	pttrn	**ulong**	R	*Signalling Pattern*. Bit coded pattern for generating the signalling output pattern.
	dly	**ulong**	R	*Repetition Delay*. Time delay, in signalling ticks, between two consecutive pattern repetitions.
	reps	**ulong**	R	*Repetitions Count*. Number of times a signalling pattern is repeated.
	tp	**ulong**	R	*Ticks Pre-scaler*. Ticks divider for creating slower patterns.

This command operates in a different way with respect to other commands. Instructions and sequences are modified individually. The way this command modifies the signalling engine parameters is controlled with one parameter named "**cmdId**".

For making modifications of one instruction the command expects the following parameters:

Dataset	Parameter	Type	Access	Description
sgnlng	cmdId	long	W	*Operation Code*. Should contain the value "0".
	idx	long	W	*Signalling Instruction Index*. Index for the signalling instruction to modify.
	inst	ulong	W	*Signalling Instruction Code*. Binary coded signalling instruction.

For making modifications of one signalling sequence definition the command expects the following parameters:

Dataset	Parameter	Type	Access	Description
sgnlng	cmdId	long	W	*Operation Code*. Should contain the value "1".
	idx	ulong	W	*Signalling Instruction Index*. Index for the signalling sequence to modify.
	pttrn	ulong	W	*Signalling Pattern*. Bit coded pattern for generating the signalling output pattern.
	dly	ulong	W	*Repetition Delay*. Time delay, in signalling ticks, between two consecutive pattern repetitions.
	reps	ulong	W	*Repetitions Count*. Number of times a signalling pattern is repeated.
	tp	ulong	W	*Ticks Pre-scaler*. Ticks divider for creating slower patterns.

4.4.12 "reset" - Global system reset command

The "**reset**" command request a global system reset.

This command is invoked without parameters and does not return any answer.

4.4.13 "scfg" - Configuration parameters persistence access command

The "**scfg**" command request the system to make permanent the configuration changes.

4.5 Sensor generated asynchronous events reference

4.5.1 "app" - Application status change notification event

The "**app**" asynchronous event is sent by the sensor to notify the occurrence of changes in the current main application status.

The dataset sent by the sensor is listed in the table below.

Dataset	Parameter	Type	Description
app	**state**	**string**	*Application State (text)*. Text representation of the current application state. Binary and text mode codes are listed on page 86.
	s	**long**	*Application State (integer)*. Binary representation of the current application state. Binary and text mode codes are listed on page 86.
	omr	**bool**	*Operating Mode Requested*. Flag indicating that the sensor received the request to enter operational mode.

4.5.2 "msg" - Informative message

The "**msg**" asynchronous event is sent by the sensor to notify of informative events in text format. These informative events are not associated to specific modules and should be ignored by the application.

The dataset sent by the sensor is listed in the table below.

Dataset	Parameter	Type	Description
msg	**tst**	**string**	*Text message*. Informative text sent by the sensor.

4.5.3 "field" - Electric field strength data encapsulating event

The "**field**" asynchronous event is periodically sent by the sensor for notifying current values of electric field strength and alarm level.

The periodicity with which the sensor sends the "**field**" event is configurable using the "**notif**" command (see section *4.4.10*).

The dataset sent by the sensor is listed in the table below.

Dataset	Parameter	Type	Description
`field`	`val`	`float`	*Electric field strength*. Electric field strength value, in units of **V/m**. This field is reported only when the sensor operates normally.
	`g`	`long`	*Current gain index*. Logical index for currently selected analog front-end gain path. Should be ignored by the application.
	`alarm`	`long`	*Current alarm level.* Currently estimated alarm level. Values range from **-1** to **3**. Value **-1** indicates the existence of abnormal conditions like errors.
	`ost`	`long`	*Internal timestamp*. Internally generated timestamp, with millisecond granularity. Can be ignored by the application.

4.5.4 "motor" - Current motor state data encapsulating event

The "`motor`" asynchronous event is periodically sent by the sensor for notifying current values of motor current consumption and speed.

The periodicity with which the sensor sends the "`motor`" event is configurable using the "`notif`" command (see section *4.4.10*).

The dataset sent by the sensor is listed in the table below.

Dataset	Parameter	Type	Description
`motor`	`imot`	`float`	*Motor current consumption.* Measure motor current consumption, in amperes.
	`rpm`	`long`	*Motor speed*. Current motor speed, in **rpm**. This value is reported only when the motor is operating.

5 Modbus RTU serial communications protocol

PREVISTORM® E-Field Sensors support two communication protocols: a proprietary plain-text command-response framing protocol (**IptP**) and **Modbus RTU**.

The **Modbus RTU** protocol is supported by firmware versions **1.030** and newer.

The **IptP** protocol is covered in section *4*. Section *4.4.3* describes the command required for switching from plain-text mode to **Modbus RTU** mode.

NOTE 1: It is advised that, once configured for operating in **Modbus RTU** mode, the sensor will not be able to communicate with the **PREVISTORM® Console**.

5.1 Serial interface

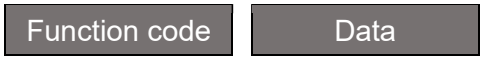
The **PREVISTORM® E-Field Sensor** serial interface conforms to **RS232** electric levels and standard. The **Modbus RTU** protocol operates over **RS232**.

Interfacing the **PREVISTORM® E-Field Sensor** to **Modbus** networks over **RS485**/**RS422** requires third party devices for making the electrical interface standard adaptation.

The serial port hardware configuration, by default, uses **8-bit words**, **No Parity**, **One Stop Bit** and **57600 bps**. Other bit rates can be configured by using the command "**`comm`**", see section *4.4.3*, or by writing to the *Modbus RTU bitrate holding register*, see section *5.4.3.5*.

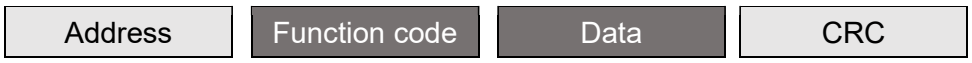
5.2 Modbus RTU framing

The **Modbus RTU** frame format is in accordance with the **Modbus Application Protocol**. The **Protocol Data Unit** (**PDU**) has following structure:



The **`Function code`** field contains the code for the function to be executed by the device. The **`Data`** field is dependent on the function code. Valid function codes, as well as supported **PDU** formats, are covered in section *5.3*, *Supported Modbus PDUs*.

For operation over serial interfaces, the **Modbus Serial Line PDU** has following structure:



The serial line **PDU** payload, fields **`Function code`** and **`Data`**, constitute the **`Message`**.

The **`Address`** field contains the **Modbus RTU** address for the device to which the frame is addressed.

The **`CRC`** field is the 16-bit **Cyclic Redundancy Check** of the **`Address`** and **`Message`** fields.

5.3 Supported Modbus PDUs

The **Modbus RTU** mode provides support for following function codes:

Code	Description
1	Read Coils (see section *5.3.2*)
3	Read Multiple Holding Registers (see section *5.3.3*).
4	Read Input Registers (see section *5.3.4*).
5	Write Single Coil (see section *5.3.5*).
6	Write Single Holding Register (see section *5.3.6*).
15	Write Multiple Coils (see section *5.3.7*).

The normal responses format is dependent on function codes and is covered in detail in subsequent sections.

5.3.1 Error response

When the sensor detects an error in the request, an error response will be sent. Error responses have following structure:

Offset	Length	Description	Values (hex)
0	Byte	Function code	81h
1	Byte	Exception code	See section *5.3.1.1*.

5.3.1.1 Exception codes

Following table summarises the exception codes supported by the sensor:

Code	Description
1	Illegal function code.
2	Illegal data address.
3	Illegal data value.

5.3.2 Read Coils (1)

Function code 1, **Read Coils**, reads between 1 and 4 coils (bits) from the **PREVISTORM® E-Field Sensor**. Each coil represents one output relay withing the address space defined in section *5.4.1*.

The request **PDU** has following structure:

Offset	Length	Description	Values (hex)
0	Byte	Function code	01h
1	Word	First coil address	0000h ~ 0003h
3	Word	Coil count	0001h ~ 0004h

The normal response has following structure:

Offset	Length	Description	Values (hex)
0	Byte	Function code	01h
1	Byte	Byte count	01h
2	Byte	Coil data	00h ~ 0Fh

The **Coil data** field contains a bits-mask with following structure:

Bit	Description
0	SIGNAL-1 output relay status.
1	SIGNAL-2 output relay status.
2	Base heater output relay status.
3	Sensor heater output relay status.

All other bits are reserved for future use and must be specified in value '0'.

A bit set to "**1**" represents an active (ON) relay state.

A bit set to "**0**" represents an inactive (OFF) relay state.

The sensor responds with an *Error response* when it detects an error in the request.

Example:

Query:

Read three coils (**0003h**) starting at Modbus address **0002h** (coil register address **0001h**).

`01 00 01 00 03`

Response:

In this example, **1** byte is returned, coils **1** and **3** are reported as active, ON (**05h**).

`01 01 05`

5.3.3 Read Multiple Holding Registers (3)

Function code `3`, **Read Multiple Holding Registers**, reads between 1 and 12 holding registers from the **PREVISTORM® E-Field Sensor**. Holding registers provide access to a subset of the sensor's configuration parameters set, see section *5.4.3*, except the *Status holding register* that represents the current configuration status.

The request **PDU** has following structure:

Offset	Length	Description	Values (hex)
0	**Byte**	Function code	03h
1	**Word**	First holding register address	0000h ~ 000Bh
3	**Word**	Register count	0001h ~ 000Ch

The normal response has following structure:

Offset	Length	Description	Values (hex)
0	**Byte**	Function code	03h
1	**Byte**	Byte count (N)	Register count * 2
2	**N Bytes**	Registers data	…

The sensor responds with an *Error response* when it detects an error in the request.

Example:

Query:

Read four (`0004h`) holding registers starting at Modbus address **`40001h`** (holding register address `0000h`).

```
03  00 00  00 04
```

Response:

The response data reports following values:

Configuration status: **Modified** (`0001h`).
Current communications protocol: **Modbus RTU** (`0001h`).
Modbus address: **0Bh** (`000Bh`).
Modbus bitrate: **57600bps** (`E100h`)

```
03  08  00 01  00 01  00 0B  E1 00
```

5.3.4 Read Input Registers (4)

Function code **4**, **Read Input Registers**, reads between 1 and 17 input registers from the **PREVISTORM® E-Field Sensor**. Input registers provide access to several status indicators for current sensor's state, see section *5.4.2*.

The request **PDU** has following structure:

Offset	Length	Description	Values (hex)
0	**Byte**	Function code	04h
1	**Word**	First input register address	0000h ~ 0010h
3	**Word**	Register count	0001h ~ 0011h

The normal response has following structure:

Offset	Length	Description	Values (hex)
0	**Byte**	Function code	04h
1	**Byte**	Byte count (N)	Register count * 2
2	**N Bytes**	Registers data	…

The sensor responds with an *Error response* when it detects an error in the request.

Example:

Query:

Read two (**0002h**) input registers starting at Modbus address **30005h** (input register address **0004h**).

04 **00 04** **00 02**

Response:

The response data reports following values:

Electric Field Register Status: **Valid** (**0001h**).
Current Alarm Level: **No Alarm Active** (**0000h**).

04 **08** **00 01** **00 00**

5.3.5 Write Single Coil (5)

Function code `5`, **Write Single Coil**, forces a single coil to ON or OFF in the **PREVISTORM® E-Field Sensor**. Each coil represents one output relay withing the address space defined in section *5.4.1*.

Writing `FF00h` will force the coil to state **ON**. Writing `0000h` will force it to state **OFF**.

The request **PDU** has following structure:

Offset	Length	Description	Values (hex)
0	Byte	Function code	05h
1	Word	Coil address	0000h ~ 0003h
3	Word	Value to force	0000h | FF00h

The normal response is an echo to the query:

Offset	Length	Description	Values (hex)
0	Byte	Function code	05h
1	Word	Coil address	0000h ~ 0003h
3	Word	Value to force	0000h | FF00h

Example:

Query:

Force the **SIGNAL-1** (`0000h`) output relay to **ON** (`FF00h`).

```
05 00 00 FF 00
```

Response:

In this example, the normal response is an echo to the query:

```
05 00 00 FF 00
```

5.3.6 Write Single Holding Register (6)

Function code `6`, **Write Single Holding Register**, writes a value to one single holding register in the **PREVISTORM® E-Field Sensor**. Holding registers provide access to a subset of the sensor's configuration parameters set, see section *5.4.3*, except the *Status holding register* that represents the current configuration status.

The request **PDU** has following structure:

Offset	Length	Description	Values (hex)
0	**Byte**	Function code	06h
1	**Word**	Holding register address	0000h ~ 000Bh
3	**Word**	Value to wite	See section *5.4.3*.

The normal response is an echo to the query:

Offset	Length	Description	Values (hex)
0	**Byte**	Function code	03h
1	**Word**	Holding register address	0000h ~ 000Bh
3	**Word**	Value to wite	See section *5.4.3*.

The sensor responds with an *Error response* when it detects an error in the request.

Example:

Query:

Configure the communications protocol register (`0001h`) for returning to Plain Text communications protocol (`0000h`).

```
03  00 00  00 00
```

Response:

In this example, the normal response is an echo to the query:

```
03  00 00  00 00
```

5.3.7 Write Multiple Coils (15)

Function code `15`, **Write Multiple Coils**, writes between 1 and 4 coils in the **PREVISTORM® E-Field Sensor**. Each coil represents one output relay withing the address space defined in section *5.4.1*.

The request **PDU** has following structure:

Offset	Length	Description	Values (hex)
0	Byte	Function code	0Fh
1	Word	First coil address	0000h ~ 0003h
3	Word	Coils count	0001h ~ 0004h
5	Byte	Byte count (N)	0001h
6	Byte	Bitmask of values to write	00h ~ 0Fh

The normal response contains the function code, starting address, and quantity of coils written to:

Offset	Length	Description	Values (hex)
0	Byte	Function code	0Fh
1	Word	First coil address	0000h ~ 0003h
3	Word	Coils count	0001h ~ 0004h

The sensor responds with an *Error response* when it detects an error in the request.

Example:

Query:

Force two (00 02h) output relays, **SIGNAL-1** (0000h) and **SIGNAL-2**, to **ON** (03h).

```
0F 00 00 00 02 01 03
```

Response:

In this example, the normal response contains the function code, starting address, and quantity of coils written to:

```
0F 00 00 00 02
```

5.4 Registers map

5.4.1 Coils

Following table summarises the coil addresses supported by the **PREVISTORM® E-Field Sensor**.

Register Address	Modbus Address	R/W	Description
0000h	1	R/W	Signalling output SIGNAL-1 relay.
0001h	2	R/W	Signalling output SIGNAL-2 relay.
0002h	3	R/W	Base heater control relay.
0003h	4	R/W	Sensor heater control relay.

NOTE 1: The **Base heater** and **Sensor heater** relays are only available in sensor models supporting de-icing.

NOTE 2: It is advised that modifying the state of the **Base heater** and **Sensor heater** relays can conduct to permanent system damages if not done properly. These relays are intended to be controlled by the built-in temperature controller.

NOTE 3: When the sensor signalling control script is configured, modifying the SIGNAL-1 or SIGNAL-2 relays can conduct to an unpredictable or erratic behaviour.

5.4.2 Input registers

Following table summarises the input register addresses supported by the **PREVISTORM® E-Field Sensor**. Each input register contains a 16-bit word.

Register Address	Modbus Address	R/W	Description
0000h	30001	R	*Device Id input register*, LSB word.
0001h	30002	R	*Device Id input register*, MSB word.
0002h	30003	R	*Firmware version input register*.
0003h	30004	R	*Bootloader footprint*.
0004h	30005	R	*Application state input register*.
0005h	30006	R	*Electric field reading registers status*.
0006h	30007	R	*E-field based alarm level*.
0007h	30008	R	*Last measured e-field intensity value (16-bit)*.
0008h	30009	R	*Last measured e-field intensity value (lo-word)*.
0009h	30010	R	*Last measured e-field intensity value (hi-word)*.
000Ah	30011	R	*Last e-field measurement timestamp (lo-word)*.
000Bh	30012	R	*Last e-field measurement timestamp (hi-word)*.
000Ch	30013	R	*Motor status input register*.
000Dh	30014	R	*Motor current consumption input register*.
000Eh	30015	R	*Motor speed input register*.
000Fh	30016	R	*CPU temperature input register*.

Register Address	Modbus Address	R/W	Description
0010h	30017	R	*Sensor's base temperature input register*.
0011h	30018	R	*Sensor's sensing plate temperature input register*.
0012h – 0015h	30019 – 30022	R	*Sensor's serial number*.
0016h – 0019h	30023 – 30026	R	*Sensor model*.
001Ah	30027	R	*Manufacture time*.
001Bh	30028	R	*Manufacture date*.
001Ch	30029	R	*Calibration time*.
001Dh	30030	R	*Calibration date*.

5.4.2.1 Device Id input register

The **Device Id** input registers contain a 32-bit device identification code. These registers identify the function of the device according to following table:

Device Id (LSB-MSB)	Device type
5045h-4653h	**PREVISTORM® E-Field Sensor**

5.4.2.2 Firmware version input register

The **Firmware version** input register contains a 16-bit unsigned value identifying the current firmware version of the device. The firmware version code is composed of two parts: Major version number (8 MSB bits) and Minor version number (8 LSB bits). The value **011E**, hex, corresponds to firmware version **1.030**.

5.4.2.3 Bootloader footprint

The **Application state** input register contains a 16-bit unsigned code that represents the current device's bootloader version.

The value in this register is used by the **PREVISTORM® Viewer** application for some functions.

5.4.2.4 Application state input register

The **Application state** input register contains a 16-bit unsigned code that represents the current application state of the sensor. Following table summarises the application states reported in this register:

Value	Description
0	System initialization in progress.
1	System is idling.
2, 3, 4	System is running internal calibrations.
5	Pausing before starting the sensor.
6	Starting the sensor.
7	Sensor is running.
8	Hardware error condition.

5.4.2.5 Electric field reading registers status

The **Electric field reading registers status** input register contains a 16-bit unsigned code that indicates whether are valid or not the values currently held in the **Electric field reading registers**. Following table summarises the value states reported in this register:

Value	Description
0	The values contained in the Electric field reading registers are INVALID.
1	The values contained in the Electric field reading registers are VALID.

Reading this register will cache the current values of e-field intensity, alarm level, and internal timestamp of last measurement. The cached values can be read from registers *Last measured e-field intensity value (16-bit)*, *Last measured e-field intensity value (lo-word)*, and *Last measured e-field intensity value (hi-word)*, *E-field based alarm level*, *Last e-field measurement timestamp (lo-word)* and *Last e-field measurement timestamp (hi-word)*.

5.4.2.6 Last measured e-field intensity value (16-bit)

The **Last measured e-field intensity value (16-bit)** input register contains the last e-field measurement result as a 16-bit signed value in **V/m**.

The values stored in this register are limited to the range from **-32768 V/m** to **32767 V/m**.

The value in this register is updated when the input register *Electric field reading registers status* is read.

The value in this register is valid only when the *Electric field reading registers status* input register contains the value **1**.

5.4.2.7 Last measured e-field intensity value (lo-word)

The **Last measured e-field intensity value (lo-word)** input register contains the 16 least-significant bits from a 32-bit signed integer representing the last measured e-field intensity.

The e-field value, represented as a 32-bit signed integer, is obtained by left-shifting 16 bits the value read from the **Last measured e-field intensity value (hi-word)** input register, and then OR-ing the result with the value read from the **Last measured e-field intensity value (hi-word)** input register.

Final e-field values are limited to the range from **-2147483648 V/m** to **2147483647 V/m**.

The value in this register is updated when the input register *Electric field reading registers status* is read.

The value in this register is valid only when the *Electric field reading registers status* input register contains the value **1**.

5.4.2.8 Last measured e-field intensity value (hi-word)

The **Last measured e-field intensity value (hi-word)** input register contains the 16 most-significant bits from a 32-bit signed integer representing the last measured e-field intensity.

The e-field value, represented as a 32-bit signed integer, is obtained by left-shifting 16 bits the value read from the **Last measured e-field intensity value (hi-word)** input register, and then OR-ing the result with the value read from the **Last measured e-field intensity value (hi-word)** input register.

Final e-field values are limited to the range from **-2147483648 V/m** to **2147483647 V/m**.

The value in this register is updated when the input register *Electric field reading registers status* is read.

The value in this register is valid only when the *Electric field reading registers status* input register contains the value **1**.

5.4.2.9 E-field based alarm level

The **E-field based alarm level** input register contains a 16-bit unsigned code representing the current alarm level reported by the sensor. Following table summarises the alarm levels reported in this register:

Value	Alarm level
0000h	Alarm level: **NO_ALARM**.
0001h	Alarm level: **ALARM-1**.
0002h	Alarm level: **ALARM-2**.
0003h	Alarm level: **ALARM-3**.
FFFFh	Alarm level: **INVALID**.

The value in this register is updated when the input register *Electric field reading registers status* is read.

The value in this register is valid only when the *Electric field reading registers status* input register contains the value **1**.

5.4.2.10 Last e-field measurement timestamp (lo-word)

The **Last e-field measurement timestamp (lo-word)** input register contains the 16 least-significant bits from a locally generated 32-bit timestamp with a one-millisecond resolution.

The value in this register is updated when the input register *Electric field reading registers status* is read.

The value in this register is valid only when the *Electric field reading registers status* input register contains the value **1**.

5.4.2.11 Last e-field measurement timestamp (hi-word)

The **Last e-field measurement timestamp (hi-word)** input register contains the 16 most-significant bits from a locally generated 32-bit timestamp with a one-millisecond resolution.

The timestamp value, represented as a 32-bit unsigned integer, is obtained by left-shifting the hi-word value 16 bits, and then OR-ing the result with the lo-word value.

Timestamp values are limited to the range from **0ms** to **4294967295ms**.

The value in this register is updated when the input register *Electric field reading registers status* is read.

The value in this register is valid only when the *Electric field reading registers status* input register contains the value **1**.

5.4.2.12 Motor status input register

The **Motor status** input register contains a 16-bit unsigned code that represents the current motor's operational state. This register is supported for diagnostics purposes only.

Following table summarises the operational states reported in this register:

Value	Description
0	The motor is idling.
1, 2	The motor is accelerating.
3	The motor is running normally.
4	The motor is decelerating.
5	The motor is in over-current condition.
6	The motor is in power-error condition.
7	The motor is in a generic error condition.

5.4.2.13 Motor current consumption input register

The **Motor current consumption** input register contains a 16-bit unsigned value indicating its actual current consumption in **mA**. This register is supported for diagnostics purposes only. The value stored in this register is updated once per second.

5.4.2.14 Motor speed input register

The **Motor speed** input register contains a 16-bit unsigned value indicating its current speed in **rpm**. This register is supported for diagnostics purposes only. The value stored in this register is updated once per second.

5.4.2.15 CPU temperature input register

The **CPU temperature** input register contains a 16-bit signed value indicating the CPU's temperature in tenths of **°C**. This register is supported for diagnostics purposes only and could have a measurement error of up to ±5ºC. The value stored in this register is updated once per second.

5.4.2.16 Sensor's base temperature input register

The **Sensor's base temperature** input register contains a 16-bit signed value indicating the sensor's base temperature in tenths of **°C**. This register is supported for diagnostics purposes only. It represents the temperature at a specific point in the sensor's base. It does not represent the exterior temperature and will vary when the de-icing circuit is operating. The value stored in this register is updated once per second.

This register has valid values only for sensor models supporting de-icing.

5.4.2.17 Sensor's sensing plate temperature input register

The **Sensor's sensing plate temperature** input register contains a 16-bit signed value indicating the sensor's sensing plate temperature in tenths of **°C**. This register is supported for diagnostics purposes only. It represents the temperature at a specific point in the sensor's sensing plate. It does not represent the exterior temperature and will vary when the de-icing circuit is operating. The value stored in this register is updated once per second.

This register has valid values only for sensor models supporting de-icing.

5.4.2.18 Sensor's serial number

The **Sensor's serial number** input registers set contains an 8 characters string that represents the sensor's serial number. Each register encodes two characters. The serial number can be up to 8 characters long. For strings shorter than 8 characters, the **NULL** (**0**) character is used as string terminator.

Following table describes how to reconstruct the serial number from data sent by the sensor:

Register	Value	Character
+0	8 bits LSB 8 bits MSB	0 1
+1	8 bits LSB 8 bits MSB	2 3
+2	8 bits LSB 8 bits MSB	4 5
+3	8 bits LSB 8 bits MSB	6 7

5.4.2.19 Sensor model

The **Sensor model name** input registers set contains an 8 characters string that represents the sensor model name. Each register encodes two characters. The sensor model name can be up to 8 characters long. For strings shorter than 8 characters, the **NULL** (0) character is used as string terminator.

Following table describes how to reconstruct the sensor model name from data sent by the sensor:

Register	Value	Character
+0	8 bits LSB 8 bits MSB	0 1
+1	8 bits LSB 8 bits MSB	2 3
+2	8 bits LSB 8 bits MSB	4 5
+3	8 bits LSB 8 bits MSB	6 7

5.4.2.20 Manufacture time

The **Manufacture time** input register contains a 16-bits unsigned value that represents the sensor manufacture time.

Following table summarises the fields contained in the **Manufacture time** input register:

Bit	Length	Field
0	5	Hour.
5	6	Minutes.
11	5	Seconds (divided by 2).

5.4.2.21 Manufacture date

The **Manufacture date** input register contains a 16-bits unsigned value that represents the sensor manufacture date.

Following table summarises the fields contained in the **Manufacture date** input register:

Bit	Length	Field
0	7	Year since 1980.
7	4	Month.
11	5	Day.

5.4.2.22 Calibration time

The **Calibration time** input register contains a 16-bits unsigned value that represents the sensor calibration time.

Following table summarises the fields contained in the **Calibration time** input register:

Bit	Length	Field
0	5	Hour.
5	6	Minutes.
11	5	Seconds (divided by 2).

The sensor returns value `0000h` from the **Calibration time** input register when the calibration is not valid.

5.4.2.23 Calibration date

The **Calibration date** input register contains a 16-bits unsigned value that represents the sensor calibration time.

Following table summarises the fields contained in the **Calibration date** input register:

Bit	Length	Field
0	5	Year since 1980.
7	6	Month.
11	5	Day.

The sensor returns value `0000h` from the **Calibration date** input register when the calibration is not valid.

5.4.3 Holding registers

Following table summarises the holding register addresses supported by the **PREVISTORM® E-Field Sensor**. Each holding register contains a 16-bit word.

Register Address	Modbus Address	R/W	Description
0000h	40001	R/W	*Status holding register*.
0001h	40002	R/W	*Communications protocol holding register*.
0002h	40003	R/W	*IptP protocol control options holding register*.
0003h	40004	R/W	*Modbus RTU address holding register*.
0004h	40005	R/W	*Modbus RTU bitrate holding register*.
0005h	40006	R/W	*Form factor holding register (lo-word)*.
0006h	40007	R/W	*Form factor holding register (hi-word)*.
0007h	40008	R/W	*Alarm LEVEL_1 threshold*.
0008h	40009	R/W	*Alarm LEVEL_2 threshold*.
0009h	40010	R/W	*Alarm LEVEL_3 threshold*.
000Ah	40011	R/W	*Alarm LEVEL_1 dwell time*.
000Bh	40012	R/W	*Alarm LEVEL_2 dwell time*.
000Ch	40013	R/W	*Alarm LEVEL_3 dwell time*.
000Dh	40014	R/W	*Alarms activation delay (lo-word)*.
000Eh	40015	R/W	*Alarms activation delay (hi-word)*.

5.4.3.1 Status holding register

The **Config status** holding register, when read, returns a 16-bit configuration status flag according to following table:

Value	Description
0	The sensor's configuration has not been changed.
1	There sensor's configuration has changes that have not been persisted yet.

When written to, the **Config status** holding register allows for following operations:

Value	Operation
`7363h`	Store the current configuration.
`8373h`	Reboot the sensor. The sensor will reboot within an interval of `125ms` to `400ms`.
`9383h`	Reset current alarm level.
`1323h`	Pause the sensor.
`1324h`	Start the sensor.
-	The rest of the values are reserved.

5.4.3.2 Communications protocol holding register

The **Communications protocol** holding register contains a 16-bit unsigned code that represents the selected communications protocol to use by the sensor. Following table summarises the communication protocols supported in this register:

Value	Description
0	Plain-text serial.
1	Modbus RTU.

Changes made to the communications protocol configuration will take effect on next sensor boot.

Changes made to this register are temporary until the sensor's configuration is persisted (see section *5.4.3.1*).

5.4.3.3 IptP protocol control options holding register

The **IptP protocol control options holding register** contains the **IptP** protocol control options, used when operating in plain-text protocol mode (**IptP**).

The following table summarises the supported plain text protocol options:

Bit	Option
0	Event frames format. 0: Use IptP frames format, see section *4.3.1*. 1: Use fixed length frames format, see section *4.3.2*.
All other bits are reserved and should be specified with value '0'.	

5.4.3.4 Modbus RTU address holding register

The **Modbus RTU address** holding register contains the 16-bit, unsigned, device address used when the selected communications protocol is **Modbus RTU**.

Valid slave device addresses are in the range 0 ~ 247.

Changes made to this register are temporary until the sensor's configuration is persisted (see section *5.4.3.1*) and will take effect on next sensor boot.

5.4.3.5 Modbus RTU bitrate holding register

The **Modbus RTU bitrate** holding register contains the 16-bit unsigned value representing the serial port bitrate to be used when the selected communications protocol is **Modbus RTU**.

Valid serial port bitrates are summarised in following table:

Bitrate
9600
14400
19200
38400
57600

Changes made to this register are temporary until the sensor's configuration is persisted (see section *5.4.3.1*) and will take effect on next sensor boot.

5.4.3.6 Form factor holding register (lo-word)

The **Form factor** holding register contains the 16 least-significant bits from the currently configured **Form Factor** parameter, see section *2.6*.

The **Form Factor** parameter is a 32-bit, single precision, floating point number compliant with the ANSI/IEEE Std 754-2008, IEEE Standard for Binary Floating-Point Arithmetic, referred to as the IEEE 754 standard.

Changes made to this register are temporary until the sensor's configuration is persisted (see section *5.4.3.1*).

Changes made to this register are applied to current configuration when the 16 most-significant bits are written to the *Form factor holding register (hi-word)* holding register.

For updating the **Form Factor** parameter, first write the 16 least-significant bits from the new value to register *Form factor holding register (hi-word)*, then write the 16 most-significant bits to register *Form factor holding register (lo-word)*.

Valid **Form Factor** parameter are in the range `1.0E-03` ~ `1.0E+03`, positive or negative.

5.4.3.7 Form factor holding register (hi-word)

The **Form factor** holding register contains the 16 most-significant bits from the currently configured **Form Factor** parameter, see section *2.6*.

The **Form Factor** parameter is a 32-bit, single precision, floating point number compliant with the ANSI/IEEE Std 754-2008, IEEE Standard for Binary Floating-Point Arithmetic, referred to as the IEEE 754 standard.

The new **Form Factor** parameter is composed using the value previously stored in register *Form factor holding register (lo-word)* and applied immediately to current configuration.

Changes made to this register are temporary until the sensor's configuration is persisted (see section *5.4.3.1*).

For updating the **Form Factor** parameter, first write the 16 least-significant bits from the new value to register *Form factor holding register (lo-word)*, then write the 16 most-significant bits to register *Form factor holding register (hi-word)*.

Valid **Form Factor** parameter are in the range **`1.0E-03 ~ 1.0E+03`**, positive or negative.

5.4.3.8 Alarm LEVEL_1 threshold

The **Alarm LEVEL_1 threshold** holding register contains a 16-bit unsigned value representing the threshold value for the alarm level **LEVEL_1** in **`V/m`**, see section *2.2.2*.

Valid **Alarm LEVEL_1 threshold** values are in the range **`1 V/m ~ 65536 V/m`**.

Changes made to this register are applied immediately to current configuration but are kept temporary until the sensor's configuration is persisted (see section *5.4.3.1*).

5.4.3.9 Alarm LEVEL_2 threshold

The **Alarm LEVEL_2 threshold** holding register contains a 16-bit unsigned value representing the threshold value for the alarm level **LEVEL_2** in **`V/m`**, see section *2.2.2*.

Valid **Alarm LEVEL_2 threshold** values are in the range **`1 V/m ~ 65536 V/m`**.

The value written to the **Alarm LEVEL_2 threshold** holding register must also be larger than the value written to the *Alarm LEVEL_1 threshold* holding register.

Changes made to this register are applied immediately to current configuration but are kept temporary until the sensor's configuration is persisted (see section *5.4.3.1*).

5.4.3.10 Alarm LEVEL_3 threshold

The **Alarm LEVEL_3 threshold** holding register contains a 16-bit unsigned value representing the threshold value for the alarm level **LEVEL_3** in **`V/m`**, see section *2.2.2*.

Valid **Alarm LEVEL_3 threshold** values are in the range **`1 V/m ~ 65536 V/m`**.

The value written to the **Alarm LEVEL_3 threshold** holding register must also be larger than the value written to the *Alarm LEVEL_2 threshold* holding register.

Changes made to this register are applied immediately to current configuration but are kept temporary until the sensor's configuration is persisted (see section *5.4.3.1*).

5.4.3.11 Alarm LEVEL_1 dwell time

The **Alarm LEVEL_1 dwell time** holding register contains a 16-bit unsigned value representing the dwell time value for the alarm level **LEVEL_1** in **`seconds`**, see section *2.2.2.2*.

Valid **Alarm LEVEL_1 dwell time** values are in the range **`1s ~ 3600s`**.

Changes made to this register are applied immediately to current configuration but are kept temporary until the sensor's configuration is persisted (see section *5.4.3.1*).

5.4.3.12 Alarm LEVEL_2 dwell time

The **Alarm LEVEL_2 dwell time** holding register contains a 16-bit unsigned value representing the dwell time value for the alarm level **LEVEL_2** in **`seconds`**, see section *2.2.2.2*.

Valid **Alarm LEVEL_2 dwell time** values are in the range **`1s ~ 3600s`**.

Changes made to this register are applied immediately to current configuration but are kept temporary until the sensor's configuration is persisted (see section *5.4.3.1*).

5.4.3.13 Alarm LEVEL_3 dwell time

The **Alarm LEVEL_3 dwell time** holding register contains a 16-bit unsigned value representing the dwell time value for the alarm level **LEVEL_3** in **`seconds`**, see section *2.2.2.2*.

Valid **Alarm LEVEL_3 dwell time** values are in the range **`1s ~ 3600s`**.

Changes made to this register are applied immediately to current configuration but are kept temporary until the sensor's configuration is persisted (see section *5.4.3.1*).

5.4.3.14 Alarms activation delay (lo-word)

The **Alarms activation delay (lo-word)** contains the 16 least-significant bits from the currently configured **Alarms activation delay** parameter.

The **Alarms activation delay** parameter represents, in **`ms`**, the delay applied before activating the alarms when the electric field intensity increases beyond the **LEVEL**_1 alarm threshold.

The **Alarms activation delay** parameter, represented as a 32-bit unsigned integer, is obtained by left-shifting the value written to register *Alarms activation delay (hi-word)* by 16 bits, and then OR-ing the result with the value written to register *Alarms activation delay (lo-word)*.

Changes to the **Alarms activation delay** parameter are applied immediately to current configuration when the most-significant bits are written to register *Alarms activation delay (hi-word)*, but are kept temporary until the sensor's configuration is persisted (see section *5.4.3.1*).

The default value for the **Alarms activation delay** parameter is **`1 s`**.

5.4.3.15 Alarms activation delay (hi-word)

The **Alarms activation delay (hi-word)** contains the 16 most-significant bits from the currently configured **Alarms activation delay** parameter.

The **Alarms activation delay** parameter represents, in **`ms`**, the delay applied before activating the alarms when the electric field intensity increases beyond the **LEVEL**_1 alarm threshold.

The **Alarms activation delay** parameter, represented as a 32-bit unsigned integer, is obtained by left-shifting the value written to register *Alarms activation delay (hi-word)* by 16 bits, and then OR-ing the result with the value written to register *Alarms activation delay (lo-word)*.

Changes to the **Alarms activation delay** parameter are applied immediately to current configuration when the most-significant bits are written to register *Alarms activation delay (hi-word)*, but are kept temporary until the sensor's configuration is persisted (see section *5.4.3.1*).

The default value for the **Alarms activation delay** parameter is **`1 s`**.